Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №5
По дисциплине «Интеллектуальный анализ данных»
Тема: «Деревья решений»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Сугак В.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
4. Оценить точность каждой модели на тестовой выборке;
5. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 4

- `Digits`
- Распознать, какая цифра (от 0 до 9) изображена на картинке 8x8 пикселей
- **Задания:**

1. Загрузите встроенный в `scikit-learn` набор данных `digits`;
2. Разделите данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
4. Для каждой модели выведите `classification_report` (sklearn.metrics), содержащий основные метрики для каждого класса;
5. Сравните общую точность моделей и определите, какая из них лучше всего подходит для этой задачи.

Ход работы

Код программы:

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from xgboost import XGBClassifier
from catboost import CatBoostClassifier

RANDOM_STATE = 42

digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=RANDOM_STATE)

print("Размер выборки:", X.shape)
print("Обучающая:", X_train.shape, "Тестовая:", X_test.shape)

models = {
    "Одиночное дерево": DecisionTreeClassifier(random_state=RANDOM_STATE),
    "Случайный лес": RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE),
    "AdaBoost": AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE),
        n_estimators=200, learning_rate=0.5, random_state=RANDOM_STATE),
    "XGBoost": XGBClassifier(
        n_estimators=200, max_depth=4, learning_rate=0.1,
        random_state=RANDOM_STATE),
    "CatBoost": CatBoostClassifier(
        iterations=300, depth=4, learning_rate=0.1,
        random_seed=RANDOM_STATE, verbose=0),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

rows = []
matrices = {}

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train, y_train, cv=cv, scoring="accuracy")

    start = time.perf_counter()
    model.fit(X_train, y_train)
    fit_time = time.perf_counter() - start
```

```
        pred_train = model.predict(X_train)
        pred_test = np.asarray(model.predict(X_test)).reshape(-1)
        matrices[name] = confusion_matrix(y_test, pred_test)

        rows.append({
            "Модель": name,
            "Accuracy train": accuracy_score(y_train, pred_train),
            "Accuracy test": accuracy_score(y_test, pred_test),
            "Macro F1 test": f1_score(y_test, pred_test, average="macro"),
            "CV mean": cv_scores.mean(),
            "CV std": cv_scores.std(ddof=1),
            "Ошибок на тесте": int((pred_test != y_test).sum()),
            "Время обучения, с": fit_time,
        })

        print(f"\n===== {name} =====")
        print(classification_report(y_test, pred_test, digits=3))

    results = pd.DataFrame(rows).sort_values("Accuracy test", ascending=False)
    print("\nСводная таблица:")
    print(results.round(4).to_string(index=False))
    results.to_csv("digits_results.csv", index=False, encoding="utf-8-sig")

    best = results.iloc[0]["Модель"]
    print(f"\nЛучшая модель по accuracy на тесте: {best}")

    fig, ax = plt.subplots(figsize=(9, 5))
    x_pos = np.arange(len(results))
    ax.bar(x_pos - 0.2, results["Accuracy test"], 0.4, label="Test")
    ax.bar(x_pos + 0.2, results["CV mean"], 0.4, yerr=results["CV std"],
           capsize=3, label="CV на train")
    ax.set_xticks(x_pos)
    ax.set_xticklabels(results["Модель"], rotation=15)
    ax.set_ylim(0.7, 1.02)
    ax.set_ylabel("Accuracy")
    ax.set_title("Сравнение моделей на Digits")
    ax.legend()
    ax.grid(axis="y", alpha=0.3)
    plt.tight_layout()
    plt.savefig("digits_comparison.png", dpi=150)
    plt.close()

    fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    for ax, (name, cm) in zip(axes.flat, matrices.items()):
        ax.imshow(cm, cmap="Blues")
        ax.set_title(name)
        ax.set_xlabel("Предсказано")
        ax.set_ylabel("Истина")
        ax.set_xticks(range(10))
        ax.set_yticks(range(10))
        for i in range(10):
            for j in range(10):
                if cm[i, j] > 0:
                    ax.text(j, i, cm[i, j], ha="center", va="center",
                            color="white" if cm[i, j] > cm.max() / 2 else "black",
                            fontsize=7)
        axes.flat[-1].axis("off")
    plt.tight_layout()
    plt.savefig("digits_confusion.png", dpi=150)
    plt.close()

    fig, axes = plt.subplots(2, 5, figsize=(12, 5))
    for ax, img, label in zip(axes.flat, digits.images, digits.target):
        ax.imshow(img, cmap="gray_r")
        ax.set_title(f"Цифра {label}")
        ax.axis("off")
    plt.tight_layout()
    plt.savefig("digits_samples.png", dpi=150)
    plt.close()
```

Вывод программы:

```
Размер выборки: (1797, 64)
Обучающая: (1257, 64) Тестовая: (540, 64)

===== Одиночное дерево =====
              precision    recall  f1-score   support

           0      0.942     0.907     0.925        54
           1      0.867     0.709     0.780        55
           2      0.782     0.811     0.796        53
           3      0.909     0.909     0.909        55
           4      0.855     0.870     0.862        54
           5      0.889     0.873     0.881        55
           6      0.960     0.889     0.923        54
           7      0.891     0.907     0.899        54
           8      0.717     0.827     0.768        52
           9      0.746     0.815     0.779        54

    accuracy                          0.852       540
   macro avg      0.856     0.852     0.852       540
weighted avg      0.856     0.852     0.853       540
```

```
===== Случайный лес =====
              precision    recall  f1-score   support

           0      1.000     0.963     0.981        54
           1      0.932     1.000     0.965        55
           2      1.000     1.000     1.000        53
           3      0.964     0.964     0.964        55
           4      0.964     0.981     0.972        54
           5      1.000     0.982     0.991        55
           6      1.000     0.981     0.991        54
           7      0.915     1.000     0.956        54
           8      0.918     0.865     0.891        52
           9      0.961     0.907     0.933        54

    accuracy                          0.965       540
   macro avg      0.965     0.964     0.964       540
weighted avg      0.965     0.965     0.965       540
```

```
===== AdaBoost =====
              precision    recall  f1-score   support

           0      0.980     0.907     0.942        54
           1      0.906     0.873     0.889        55
           2      1.000     0.943     0.971        53
           3      0.981     0.945     0.963        55
           4      0.962     0.926     0.943        54
           5      0.964     0.964     0.964        55
           6      0.963     0.963     0.963        54
           7      0.947     1.000     0.973        54
           8      0.787     0.923     0.850        52
           9      0.891     0.907     0.899        54

    accuracy                          0.935       540
   macro avg      0.938     0.935     0.936       540
weighted avg      0.939     0.935     0.936       540
```

```
===== XGBoost =====
              precision    recall  f1-score   support

           0      0.961     0.907     0.933        54
           1      0.893     0.909     0.901        55
           2      0.981     0.981     0.981        53
           3      0.931     0.982     0.956        55
           4      0.962     0.944     0.953        54
           5      0.946     0.964     0.955        55
           6      1.000     0.981     0.991        54
           7      0.947     1.000     0.973        54
           8      0.882     0.865     0.874        52
           9      0.942     0.907     0.925        54

    accuracy                          0.944       540
   macro avg      0.945     0.944     0.944       540
weighted avg      0.945     0.944     0.944       540
```

```
===== CatBoost =====
              precision    recall  f1-score   support

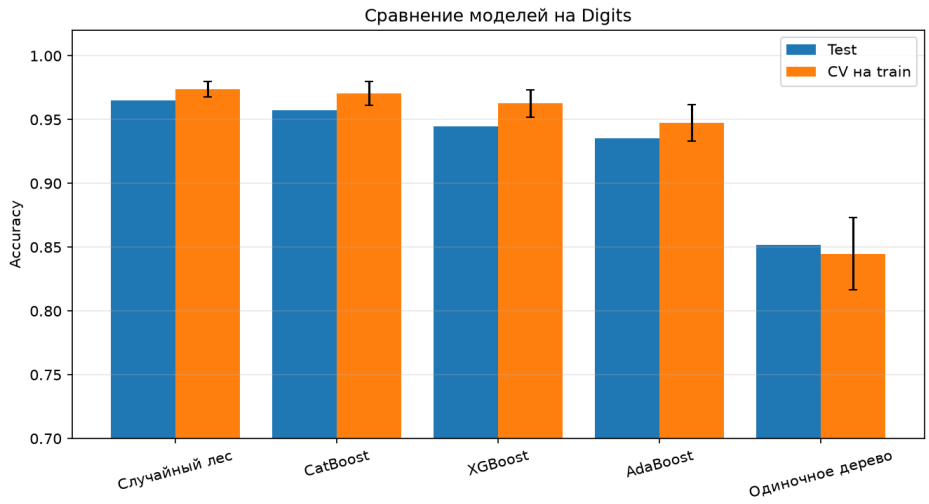
           0      1.000     0.963     0.981        54
           1      0.881     0.945     0.912        55
           2      1.000     1.000     1.000        53
           3      0.964     0.982     0.973        55
           4      0.963     0.963     0.963        54
           5      0.981     0.964     0.972        55
           6      0.981     0.981     0.981        54
           7      0.931     1.000     0.964        54
           8      0.918     0.865     0.891        52
           9      0.961     0.907     0.933        54

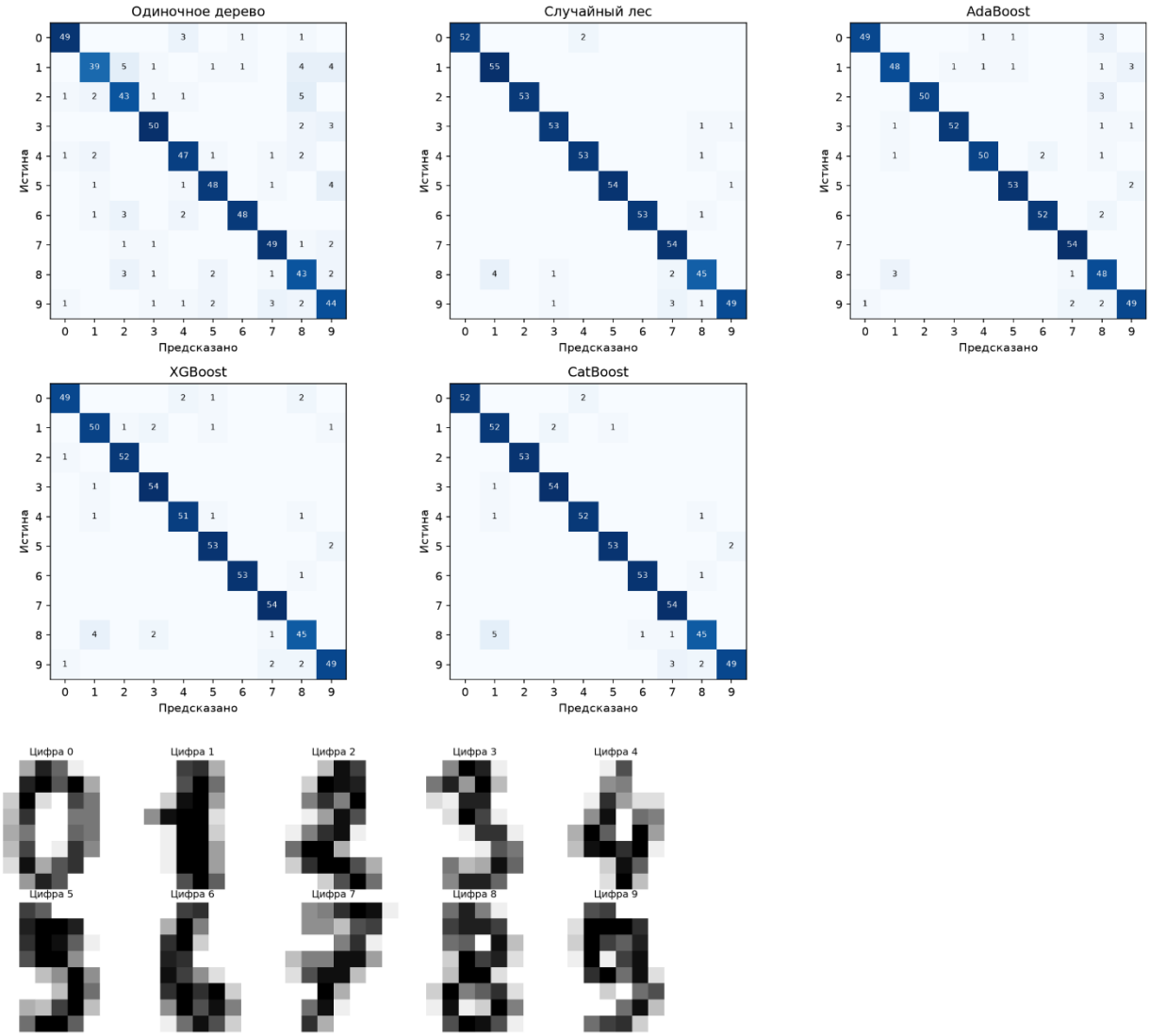
    accuracy                          0.957       540
   macro avg      0.958     0.957     0.957       540
weighted avg      0.958     0.957     0.957       540


Сводная таблица:
                Модель  Accuracy train  Accuracy test  Macro F1 test  CV mean  CV std  Ошибок на тесте  Время обучения, с
          Случайный лес             1.0         0.9648         0.9644   0.9737  0.0061               19             1.1331
               CatBoost             1.0         0.9574         0.9572   0.9706  0.0092               23             5.3383
                XGBoost             1.0         0.9444         0.9441   0.9626  0.0108               30             1.3171
               AdaBoost             1.0         0.9352         0.9357   0.9475  0.0144               35             1.8587
Одиночное дерево                    1.0         0.8519         0.8522   0.8449  0.0285               80             0.0233

Лучшая модель по accuracy на тесте: Случайный лес
```

Графики:





В работе на наборе Digits сравнивались одиночное дерево, случайный лес, AdaBoost, XGBoost и CatBoost. Лучший результат показал случайный лес: accuracy на тесте 0.965 и macro F1 0.964 (19 ошибок из 540), затем CatBoost 0.957, XGBoost 0.944, AdaBoost 0.935. Одиночное дерево заметно слабее: accuracy 0.852 и 80 ошибок, при этом на обучающей выборке все модели дают 1.0, то есть дерево сильно переобучается. Кросс-валидация подтверждает порядок моделей (лес 0.974, CatBoost 0.971, дерево 0.845), а разница между лесом и CatBoost меньше разброса между фолдами. Хуже всего распознаются цифры 8 и 9, лучше всего 2 и 6. Для этой задачи лучше всего подходит случайный лес: он дает наивысшую точность при времени обучения около 1 секунды, а CatBoost и XGBoost близки к нему по качеству.

Вывод: научилась осуществлять предобучение нейронных сетей с помощью RBM.